

UTBM - Semestre Automne 2024

Rapport Projet LO21

Réseaux de neurones

Mathis Richard
Jad Kahlaoui
12/11/2024

Table des matières

1. Description des choix de conception et d'implémentation relatifs aux structures de données utilisées et à la démarche adoptée.....	2
1.1. Choix des structures de données.....	2
1.1.1. Types de structures de données utilisées	2
1.1.2. Critères de sélection.....	2
1.2. Démarche adoptée.....	2
2. Algorithmes des sous-programmes et leurs explications	3
2.1. InitNeur	3
2.2. OutNeurone	4
2.3. InitCouche	4
2.4. OutCouche	5
2.5. CreerResNeur	5
2.6. PropagationAvant	6
3. Jeux d'essais	6
3.1. Réseau ET	6
3.2. Réseau OU.....	7
3.3. Réseau NOT	7
3.4. Réseau multi-couches (A ET (non B) ET C) OU (A ET (non C))	8

1. Description des choix de conception et d'implémentation relatifs aux structures de données utilisées et à la démarche adoptée

Lors de la conception et de l'implémentation d'un système logiciel, les choix des structures de données et de la démarche adoptée jouent un rôle fondamental dans la performance, la lisibilité et la maintenabilité du code. Voici une description structurée des aspects à aborder :

1.1. Choix des structures de données

Le choix des structures de données est dicté par les besoins spécifiques du problème, notamment en termes de :

- **Efficacité temporelle** (temps d'exécution pour les opérations courantes : insertion, suppression, recherche).
- **Efficacité spatiale** (gestion de la mémoire).

1.1.1. Types de structures de données utilisées

- **Tableaux** : utilisés pour des accès rapides en fonction d'un index lorsque les données ont une taille fixe.
- **Listes chaînées** : préférées dans des situations nécessitant des insertions et suppressions fréquentes sans coût de déplacement.

1.1.2. Critères de sélection

- **Flexibilité** : adoption de listes chaînées lorsque la taille des données est dynamique.

1.2. Démarche adoptée

Par rapport à ce qui nous est demandé dans le sujet, les structures de données sont choisies en tenant compte des compromis entre complexité, performance et facilité d'implémentation. La démarche adoptée suit une approche incrémentale et itérative, avec un accent sur l'analyse des besoins et les tests pour valider les choix techniques.

Il a donc été réfléchi entre le choix de tableaux ou de liste chaînées. Comme expliqué plus tôt, un tableau permet de traiter un nombre fixe de donnée, à l'inverse des listes chaînées, ou leur taille est dynamique et sans contrainte.

Il a donc été convenu que :

- Les poids sont une structure représentée par la valeur de leur masse, mais aussi par le poids qui suit un certain poids. En effet, cela revient à dire que les listes de poids (qui appartiennent à un neurone) sont des listes chaînées. En effet, d'un neurone à l'autre, le nombre d'entrée ne sera pas le même, ce qui fait que le nombre de poids ne sera pas le même non plus, et donc ne sera pas statique. Le choix naturel était donc d'en dresser une liste chaînée.
- Les neurones sont une structure représentée par la valeur de leur seuil, leur nombre d'entrée, la liste de poids, mais aussi par le neurone qui suit un certain neurone. En effet, cela revient à dire que les listes de neurones (qui forment une couche) sont des listes chaînées. En effet, d'une couche à l'autre, le nombre de neurones ne sera pas le même, et donc ne sera pas statique. Le choix naturel était donc d'en dresser une liste chaînée.
- Les couches sont une structure représentée par la valeur de leur nombre de neurones, la liste de neurones, mais aussi par la couche qui suit une certaine couche. En effet, cela revient à dire que les listes de couches (qui forment un réseau) sont des listes chaînées. En effet, d'un réseau à l'autre, le nombre de couches ne sera pas le même, et donc ne sera pas statique. Le choix naturel était donc d'en dresser une liste chaînée.

Pour le reste, lorsqu'on fournit une liste d'entiers (correspondant aux entrées) à une fonction, celle-ci est sous forme de tableau. Par ailleurs, la sortie d'une couche, est produite sous forme de tableau, car elle sera l'entrée de la couche d'après, qui est demandé sous forme de tableau.

2. Algorithmes des sous-programmes et leurs explications

2.1. InitNeur

Fonction InitNeur(nbEntrees : Entier) : Neurone

Début

listePoids <- listeVide()

Pour i allant de 1 à nbEntrees **faire**

 mass <- demander('Renseignez le i ème poids')

```

newPoids : Poids <- créerElement()
mass(newPoids) <- mass
listePoids <- ajouterQueue(listePoids, newPoids)

```

Fin Pour

```

seuil <- demander('Veuillez définir le seuil du neurone)
newNeurone : Neurone <- créerElement()
seuil(newNeurone) <- seuil
poids(newNeurone) <- listePoids
nbEntrees(newNeurone) <- nbEntrees

```

```

InitNeur(nbEntrees) <- newNeurone

```

Fin

2.2. OutNeurone

Fonction OutNeurone(neurone : Neurone , listeEntiers : Tableau d'entier) : Entier

Début

```

Size : Entier <- nbEntrees(neurone)
Somme : Entier <- 0
tmp : Poids <- poids(neurone)

```

Pour i allant de 0 a Size-1 faire

```

    Somme <- Somme + listeEntier[i] x mass(tmp) // Le x represente un multiplie
    tmp <- succ(tmp)

```

Fin Pour

Si somme >= seuil(neurone) **alors**

```

    OutNeurone <- 1

```

Sinon

```

    OutNeurone <- 0

```

Fin SI

Fin

2.3. InitCouche

Fonction InitCouche(nbEntrees : Entier , nbNeurones : Entier) : Couche

Début

```

listeNeurones <- listeVide()
tmp : Neurone

```

Pour i allant de 0 à nbNeurones-1 Faire

```

    Ecrire(« i + 1 »eme Neuronne )
    newNeuronne : Neurone <- InitNeur(nbEntrees)

```

```

succ(newNeuronne) <- NULL
Si i = 0 Alors
    listeNeurones <- newNeuronne
Sinon
    succ(tmp) <- newNeuronne
Fin Si
    tmp <- newNeuronne
Fin Pour
couche : Couche <- créerElement()
succ(couche) <- NULL
neurone(couche) <- listeNeurones
nbNeurones(couche) <- nbNeurones
InitCouche <- couche
Fin

```

2.4. OutCouche

Procédure OutCouche(couche : Couche, listeEntier : Liste d'entiers , listeSortie : Liste d'entiers)

Début

```

tmp : Neurone <- neurone(couche)
Pour i allant de 0 à nbNeurone(couche) - 1 faire
    listeSortie[i] <- OutNeurone(tmp,listeEntier)
    tmp <- succ(tmp)

```

Fin Pour

Fin

2.5. CreerResNeur

Fonction CreerResNeur(nbCouche : Entier , listeNbNeuroneParCouche : Tableau d'entier) : Couche

Début

```

listeCouches : <- listeVide()
tmp : liste de Couche
Pour i allant de 0 à nbCouche-1 faire
    Ecrire(« i+1 »eme Couche)
    newCouche : liste de Couche

```

Si i = 0 **faire**

```

    nbEntrees <- Demander(Quel est le nombre d'entrées premiere couche ?)
    newCouche <- InitCouche(listeNbNeuroneParCouche[0],nbEntrees)

```

```

    listeCouches <- newCouche
sinon
    newCouche <-
InitCouche(listeNbNeuroneParCouche[i],listeNbNeuroneParCouche[i-1])
    succ(tmp) <- newCouche
Fin Si
    Succ(newCouche)<-INDEFINI
    Tmp <- NewCouche
Fin Pour
    CreerResNeur <- listeCouches
Fin

```

2.6. PropagationAvant

Procédure PropagationAvant(reseau : Couche, listeEntrees[] : Tableau d'entier)

Début

```

premiere_couche : Couche <- reseau
listeSortiesNeuronesPrev : Liste d'entiers <- créerListeVide()
OutCouche(premiere_couche,listeEntrees,listeSortiesNeureonesPrev)
couche : Couche <- succ(premiere_couche)

```

Tant que NonVide(couche) **faire**

```

    listeSortiesNeurones : Liste d'entiers <- créerListeVide()
    OutCouche(couche, listeSortiesNeureonesPrev ,listeSortiesNeureones)
    listeSortiesNeuronesPrev <- listeSortiesNeurones

```

Si estvide(succ(couche) **alors**

Pour i allant de 0 à nbNeurone(couche) -1 **faire**

 Ecrire(Sortie du réseau « i+1 » : « listeSortiesNeurones[i] »)

Fin Pour

Fin Si

Couche<-succ(couche)

Fin Tant que

Fin

3. Jeux d'essais

3.1. Réseau ET

Configuration :

- Un seul neurone avec n entrées

- Un poids de 1 pour chaque entrée
- Le seuil est fixé à n (= nombre d'entrées)

Interprétation :

- La sortie sera égale à 1, si toutes les entrées sont égales à 1 (car la somme doit être supérieure ou égale à n , pour dépasser le seuil), sinon elle vaut 0.

Essais :

- Entrées [1,1] -> Sortie = 1
- Entrées [1,0] ou [0,1] ou [0,0] -> Sortie = 0

3.2. Réseau OU

Configuration :

- Un seul neurone avec n entrées
- Un poids de 1 à chaque entrée
- Le seuil est fixé à 1

Interprétation :

- La sortie sera égale à 1, si au moins une entrée est égale à 1 (car la somme doit être supérieure ou égale à 1, pour dépasser le seuil), sinon elle vaut 0.

Essais :

- Entrées [1,0] ou [0,1] ou [1,1] -> Sortie = 1
- Entrées [0,0] -> Sortie = 0

3.3. Réseau NOT

Configuration :

- Un seul neurone avec une seule entrée
- Un poids de -1 pour l'entrée
- Le seuil est fixé à 0

Interprétation :

- La sortie sera égale à 1, si l'entrée est égale à 0 (car 0 est supérieur ou égal au seuil), sinon elle vaut 0 si l'entrée vaut 1. C'est donc l'entrée inversée.

Essais :

- Entrée [0] -> Sortie = 1
- Entrée [1] -> Sortie = 0

3.4. Réseau multi-couches (A ET (non B) ET C) OU (A ET (non C))

Configuration :

- 3 couches

Première couche :

- Premier neurone (identité) avec une liste de poids égale à $[1,0,0]$ et un seuil fixé à 1
- Deuxième neurone (NOT) avec une liste de poids égale à $[0,-1,0]$ et un seuil fixé à 0
- Troisième neurone (identité) avec une liste de poids égale à $[0,0,1]$ et un seuil fixé à 1
- Quatrième neurone (NOT) avec une liste de poids égale à $[0,0,-1]$ et un seuil fixé à 0

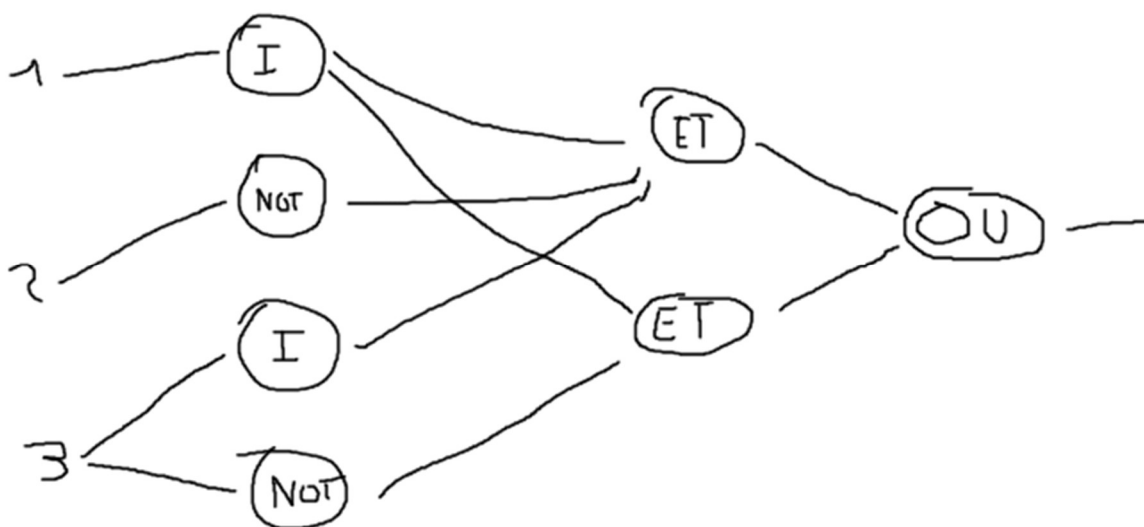
Deuxième couche :

- Premier neurone (ET) avec une liste de poids égale à $[1,1,1,0]$ et un seuil fixé à 3
- Deuxième neurone (ET) avec une liste de poids égale à $[1,0,0,1]$ et un seuil fixé à 2

Troisième couche :

- Seuil et unique neurone (OU) avec une liste de poids égale à $[1,1]$ et un seuil fixé à 1

Illustration :



Interprétation :

Soit [A,B,C] l'entrée sur le réseau multi-couches

Alors interprétons les 8 cas différents, que l'on devrait obtenir lors des essais

- [1,0,0] : (VRAI ET (non FAUX) ET FAUX) OU (VRAI ET (non FAUX)) = (VRAI ET VRAI ET FAUX) OU (VRAI ET VRAI) = FAUX OU VRAI = VRAI -> Sortie = 1
- [0,1,0] : (FAUX ET (non VRAI) ET FAUX) OU (FAUX ET (non FAUX)) = (FAUX ET FAUX ET FAUX) OU (FAUX ET VRAI) = FAUX OU FAUX = FAUX -> Sortie = 0
- [0,0,1] : (FAUX ET (non FAUX) ET VRAI) OU (FAUX ET (non VRAI)) = (FAUX ET VRAI ET VRAI) OU (FAUX ET FAUX) = FAUX OU FAUX = FAUX -> Sortie = 0
- [1,1,0] : (VRAI ET (non VRAI) ET FAUX) OU (VRAI ET (non FAUX)) = (VRAI ET FAUX ET FAUX) OU (VRAI ET VRAI) = FAUX OU VRAI = VRAI -> Sortie = 1
- [1,0,1] : (VRAI ET (non FAUX) ET VRAI) OU (VRAI ET (non VRAI)) = (VRAI ET VRAI ET VRAI) OU (VRAI ET FAUX) = VRAI OU FAUX = VRAI -> Sortie = 1
- [0,1,1] : (FAUX ET (non VRAI) ET VRAI) OU (FAUX ET (non VRAI)) = (FAUX ET FAUX ET VRAI) OU (FAUX ET FAUX) = FAUX OU FAUX = FAUX -> Sortie = 0
- [0,0,0] : (FAUX ET (non FAUX) ET FAUX) OU (FAUX ET (non FAUX)) = (FAUX ET VRAI ET FAUX) OU (FAUX ET VRAI) = FAUX OU FAUX = FAUX -> Sortie = 0
- [1,1,1] : (VRAI ET (non VRAI) ET VRAI) OU (VRAI ET (non VRAI)) = (VRAI ET FAUX ET VRAI) OU (VRAI ET FAUX) = FAUX OU FAUX = FAUX -> Sortie = 0

En résumé, seul les listes d'entrée [1,0,0], [1,1,0] et [1,0,1] devraient produire une sortie égale à 1.

Essais :

Effectuons désormais les essais en configurant les entrées vues précédemment.

En configurant les couches comme expliqué précédemment, on obtient cela :

```
---- Entrée = [1,0,0] :  
Sortie du réseau 1: 1  
---- Entrée = [0,1,0] :  
Sortie du réseau 1: 0  
---- Entrée = [0,0,1] :  
Sortie du réseau 1: 0  
---- Entrée = [1,1,0] :  
Sortie du réseau 1: 1  
---- Entrée = [1,0,1] :  
Sortie du réseau 1: 1  
---- Entrée = [0,1,1] :  
Sortie du réseau 1: 0  
---- Entrée = [0,0,0] :  
Sortie du réseau 1: 0  
---- Entrée = [1,1,1] :  
Sortie du réseau 1: 0
```

Ce qui correspond à l'interprétation.